

Betriebsdokumentation

Inhaltsverzeichnis

1. Übersicht der verwendeten Technologien	1
2. Systemvoraussetzungen	2
2.1. Hardware-Mindestanforderungen	2
2.2. Softwarevoraussetzungen	2
3. Inbetriebnahme	2
3.1. Umgebungsvariablen	2
3.2. Docker-Compose Instruktionen	4
3.3. Container Starten	4
3.4. Container Stoppen	5
3.5. Login und Browser	5
4. Kontakt - Entwicklungsteam	5

Ziel dieser Betriebsdokumentation ist es, Administratoren beim Einrichten, Konfigurieren, Betreiben und Warten unserer Anwendung zu unterstützen. Dieses Dokument beschreibt ausschließlich systemspezifische Besonderheiten. Allgemeine Software-Dokumentationen (z.B. Docker, Supabase/PostgreSQL, React, Prisma) werden bei Bedarf verlinkt.

1. Übersicht der verwendeten Technologien

Die Anwendung arbeitet über ein Mehrcontainer-System. Das heißt, die Anwendung wurde nach Schichten getrennt und in einzelne Container aufgeteilt, in welchen Prozesse losgelöst voneinander laufen. Die Trennung dient dabei der erleichterten Wartbarkeit und folgt dem Prinzip der "Separation of Concerns".

Die Containerisierung erfolgt mittels [Docker Compose](#) und erzeugt die folgenden Images bzw. Container:

- **gewohnheitstier-frontend:**
 - Web-UI als React-SPA (Create React App)
 - Schnittstellen zum Backend
 - veröffentlicht auf Port 3000 des Hostgerätes
- **gewohnheitstier-backend:**
 - Logikschicht
 - Schnittstellen zum Frontend
 - Schnittstellen zur Datenbank sowie Authentifizierung ([Supabase](#))

- veröffentlicht auf Port 3001 des Hostgerätes

2. Systemvoraussetzungen

2.1. Hardware-Mindestanforderungen

Eine Abschätzung der benötigten Ressourcen kann zum aktuellen Zeitpunkt nur durch lokales Beobachten der Container erfolgen. Eine gemittelte Messung der Aktivitäten unter Last (Mehrnutzer-Betrieb) ist noch offen und könnte hier zu genaueren Angaben beitragen. Die folgende Abschätzung benötigter Hardwareressourcen ist daher keine untere Schranke. Die Anwendung kann unter Umständen auf geringeren Spezifikationen ausgeführt werden, jedoch können wir nicht dafür garantieren. Daher empfehlen wir folgende Spezifikationen:

- 64-bit Architektur
- Multicore-CPU oder 2vCPU (>1GHz)
- KVM Virtualisierung unterstützt
- min. 4GB RAM
- 50 GB freier Speicherplatz
- Ubuntu LTS 24.xx.x Desktop oder Server
- stabile, schnelle Internetverbindung

2.2. Softwarevoraussetzungen

Auf dem System muss Docker bzw. Docker Compose installiert und lauffähig sein. Dafür kann die offizielle Dokumentation herangezogen werden ([Docker](#)). Der Administrator oder Betreiber benötigt Zugriff auf das Terminal des Hostgeräts und ggf. Root-Rechte, um Portfreigaben tätigen zu dürfen sowie Konfigurationsdateien erstellen und bearbeiten zu können.

Die Anwendung ist durchaus auf anderen Betriebssystemen ausführbar, wie in etwa andere Linux-Distributionen, MacOS oder Windows. Sollten Sie Schwierigkeiten dabei haben, die Anwendung auf einem anderen Betriebssystem auszuführen, so kontaktieren Sie bitte die am Ende des Dokuments genannten Personen.

3. Inbetriebnahme

3.1. Umgebungsvariablen

Damit die Dienste innerhalb der Container auf Services der Datenbank und das Authentifizierungsverfahren zugreifen können, müssen gewisse Umgebungsvariablen gesetzt werden. Diese Variablen beinhalten zum Teil sensible Informationen, die erweiterten Zugang zur Datenbank und deren Authentifizierungsverfahren ermöglichen und sollten daher **niemals öffentlich zugänglich** gemacht werden. Bitte wenden Sie sich an das Entwicklungsteam oder den PO, um Zugang zu diesen Informationen zu erhalten, sollten Sie diese noch nicht besitzen.

Alle Umgebungsvariablen müssen in der Textdatei `.env.production` gespeichert werden. Die Variablen müssen dabei durch einen Zeilenumbruch voneinander getrennt sein und nach den folgenden Konventionen formuliert werden.

3.1.1. Database-URL

Die Database-URL ist die Adresse, unter der der Backend-Container die Datenbank abrufen. Sie ist folgendermaßen aufgebaut:

```
DATABASE_URL="postgresql://<user>.<project_reference>:<password>@aws-0-eu-central-1.pooler.supabase.com:5432/postgres"
```

`user`, `project_reference` und `password` werden Ihnen vom Entwicklungsteam bereitgestellt. Mit dieser URL haben Sie vollen Zugriff auf alle Tabellen der Datenbank, die für die Anwendung benötigt werden. Davon ausgenommen ist die Authentifizierungstabelle, in der die Nutzerdaten gespeichert werden.

3.1.2. Supabase-URL

Die Supabase-URL verbindet die Anbindung mit der API von Supabase. Diese wird für die Authentifizierung benötigt, welche losgelöst vom Gewohnheitstier über Supabase geschieht.

```
SUPABASE_URL=https://<project_reference>.supabase.co/
```

3.1.3. Supabase-Anon-Key

Der Anon-Key wird benötigt, um den eigenen Dienst bei Supabase authentifizieren zu können. Er authentifiziert Sie als rechtmäßigen Nutzer und ermöglicht somit den Zugriff auf zuvor genannte Dienste und Daten.

```
SUPABASE_ANON_KEY=****
```

3.1.4. Node-Env und Port

Node-Env und Port geben den Containern den nötigen Kontext, um in einer Produktionsumgebung zu laufen. Dabei legt Node-Env fest, welche Node Pakete des Containers zur Laufzeit aktiv sein sollen und Port bestimmt den Port, an dem das Backend zu erreichen ist. Damit nur der Produktions-build ausgeführt wird und keine Entwicklungspakete eingebunden werden, muss die folgende Konfiguration eingehalten werden:

```
NODE_ENV=production  
PORT=3001
```

3.2. Docker-Compose Instruktionen

Damit die Anwendung über Docker Compose gestartet werden kann, muss im Verzeichnis, in dem die `.env.production`-Datei liegt, eine sog. `docker-compose.yml`-Datei gespeichert sein. Diese enthält die nötigen Anweisungen bzw. den Kontext, damit Docker die Container ordnungsgemäß orchestrieren kann. Sie enthält keine sensiblen Informationen und kann wie hier dokumentiert übernommen werden:

```
services:
  backend:
    container_name: backend-prod
    image: ghcr.io/1tristan0/se-habittracker-3b/gewohnheitstier-backend:latest
    ports:
      - "3001:3001"
    env_file:
      - .env.production
    environment:
      NODE_ENV: production
    restart: unless-stopped

  frontend:
    container_name: frontend-prod
    image: ghcr.io/1tristan0/se-habittracker-3b/gewohnheitstier-frontend:latest
    ports:
      - "3000:3000"
    environment:
      REACT_APP_API_BASE_URL: http://localhost:3001
      NODE_ENV: production
    restart: unless-stopped
```

In dieser Datei werden weitere Informationen wie der Containername und welche Imageversion der Container verwendet festgelegt. Die Imageversion wird über `image: ...` definiert. `:latest` steht hier für die aktuellste Version, die im Repository unter dem dev-Branch veröffentlicht wurde und beinhaltet immer die neusten Änderungen. `:stable` bezieht das aktuellste Image, das über den main-Branch veröffentlicht wurde und ist daher die sicherste Option. Über die Angabe einer bestimmten Commit-ID, kann hier auch auf ältere oder andere neue Versionen zugegriffen werden. Schauen Sie dazu im [GitHub Container Repository](#) nach oder wenden Sie sich an das Entwicklungsteam. Sollte Ihnen keine andere spezifische Version genannt werden, so belassen Sie die Imageversion bei `:latest` oder `:stable`.

Achten Sie darauf, dass die Ports 3000, 3001 und 5432 Ihres Hostgeräts nicht bereits durch andere Anwendungen belegt sind.

3.3. Container Starten

Sind die oberen Voraussetzungen erfüllt, wechseln Sie in das Verzeichnis, in dem die beiden Dateien `.env.production` und `docker-compose.yml` liegen. Führen Sie den folgenden Befehl in der Kommandozeile aus:

```
docker compose up
```

Docker beginnt nun, nach den spezifizierten Images zu suchen, die Container zu bauen und schließlich auszuführen. Bei der ersten Ausführung müssen die Images zunächst vom GHCR geladen werden. Dies kann etwas Zeit in Anspruch nehmen.

Schließlich sollte in der Konsole zu erkennen sein, dass die beiden Container gestartet wurden. Hier werden ggf. Fehlermeldungen und Warnungen geloggt, die Sie bei Bedarf an uns weiterleiten können, falls Sie Schwierigkeiten bei der Ausführung haben sollten. Um die Containeransicht zu verlassen, drücken Sie die Taste **d** auf Ihrer Tastatur.

Die Webanwendung ist nun über <http://localhost:3000/> per Browser erreichbar.

3.4. Container Stoppen

Um die Container herunterzufahren, geben Sie folgenden Befehl in der Konsole ein:

```
docker compose down -v
```

Mit der Option **-v** entfernen Sie zusätzliche Volumes, die die Container zur Laufzeit erstellt haben.

3.5. Login und Browser

Die Anwendung funktioniert auf allen aktuellen Browserversionen von Chrome, Firefox und Safari. Zum Anlegen eines neuen Logins navigieren Sie auf die Startseite und wechseln Sie zur Ansicht "Registrieren". Ihnen wird eine Bestätigungsmail zugesendet, mit der Sie sich authentifizieren können. Die Weiterleitung führt aktuell nur auf eine leere Seite. Anschließend können Sie Ihren erstellten Account nutzen.

4. Kontakt - Entwicklungsteam

Bei Fragen wenden Sie sich bitte nach Art der Frage an die folgenden Personen:

- Containerisierung, GHCR: Tristan Weiland
- Supabase (Login, DB): Paul Händler, Tristan Weiland
- Fehlermeldungen, Logfiles: Paul Händler, Patrick Eger, Tristan Weiland